

Task 1 — Python Automation: Smart File Organizer

Objective

Develop a Flask-based automation system that enables users to upload a ZIP file, automatically organizes its contents into categorized folders, creates backups, detects duplicates, removes empty folders, renames files, and generates a downloadable organized ZIP file.

This updated version is **cloud-deployable (Render compatible)** and functions without local folder access.

Tools & Technologies

- Python 3.x
 - Flask (Web Framework)
 - OS Module
 - Shutil
 - Hashlib
 - Zipfile
 - Tempfile
 - UUID
 - HTML5, CSS3
 - Gunicorn (Deployment)
 - Render Cloud Hosting
-

Project Overview

The **Smart File Organizer** is an intelligent automation system designed to simplify file management by automatically categorizing, renaming, and cleaning up files.

This cloud-based version allows users to upload ZIP files through a web interface, process them securely in a temporary environment, and download the organized results — all without local file system access.

Core functionalities include:

- File categorization based on extensions
 - Duplicate detection using hashing
 - Bulk file renaming
 - Empty folder cleanup
 - Organized ZIP file generation
 - Cloud-safe temporary processing
-

How It Works

1. ZIP File Upload

Users upload a ZIP file containing multiple files through the web interface.

2. Temporary Workspace Creation

The system creates a secure temporary directory for processing the uploaded ZIP file.

3. Extraction Process

The uploaded ZIP file is extracted into the working directory for organization.

4. Automatic File Organization

Files are categorized automatically based on their extensions:

Category	Extensions
Images	.jpg, .jpeg, .png, .gif
Documents	.pdf, .docx, .txt
Audio	.mp3, .wav
Videos	.mp4, .avi
Code	.py, .js, .cpp

5. Bulk File Renaming

Files are renamed using a user-defined or default prefix for consistency.

Example:

Before → image1.jpg

After → FILE_image1.jpg

6. Empty Folder Cleanup

All empty directories are automatically removed after processing.

7. Duplicate File Detection

Duplicate files are detected using the **MD5 hashing algorithm** to ensure unique content.

8. Output ZIP Generation

The organized folder structure is compressed into a new downloadable ZIP file.

9. Download Result

Users can download the processed ZIP file directly from the web interface.

Key Code Snippet

```
def organize_files(folder):
    categories = {
        "Images": [".jpg", ".jpeg", ".png"],
        "Documents": [".pdf", ".docx", ".txt"],
        "Audio": [".mp3", ".wav"],
        "Videos": [".mp4", ".avi"]
    }

    for file in os.listdir(folder):
        file_path = os.path.join(folder, file)

        if os.path.isfile(file_path):
            ext = os.path.splitext(file)[1].lower()

            for category, extensions in categories.items():
                if ext in extensions:
                    target_folder = os.path.join(folder, category)
                    os.makedirs(target_folder, exist_ok=True)
                    shutil.move(file_path, os.path.join(target_folder, file))
                    break
```

Key Features

- ✓ ZIP File Upload System
 - ✓ Cloud-Safe File Processing
 - ✓ Automatic Categorization
 - ✓ Bulk Renaming System
 - ✓ Duplicate Detection (Hash-Based)
 - ✓ Empty Folder Cleanup
 - ✓ Downloadable Output ZIP
 - ✓ Temporary Workspace Isolation
 - ✓ Render Deployment Support
 - ✓ No Local System Dependency
-

Sample Input

Upload File:

project_files.zip

Contents:

image1.jpg
image2.png

report.pdf
song.mp3
video.mp4

Sample Output

Files Moved : 5
Files Renamed : 5
Empty Folders Removed: 1
Duplicates Found : 0
Download ZIP : organized_output.zip

Project Structure (Updated Cloud Version)

```
Task1-Python-Automation/  
|  
├── app.py  
├── modules/  
│   └── organizer.py  
├── templates/  
│   └── index.html  
├── static/  
│   └── style.css  
├── uploads/      (temporary)  
├── outputs/      (generated ZIP)  
├── requirements.txt  
├── Procfile  
└── README.md
```

Deployment (Render)

Start Command:

gunicorn app:app

Build Command:

pip install -r requirements.txt

Screenshots

Upload Interface



File Detection Result



Smart File Organizer



Choose File

No file chosen



Organize Now

0

Files Moved

0

Files Renamed

0

Folders Removed

1

Duplicates Found

Download Organized ZIP

Smart File Organizer

A Flask automation tool that organizes uploaded ZIP files into structured folders.

⚡ Built by Madhiyarasu D D • 🚀 Flask Project • 📁 Internship Task

© 2026 Smart File Organizer



Download ZIP Page



Smart File Organizer

Upload ZIP → Organize Files Automatically → Download Result



Choose File

No file chosen



Organize Now



Organization Completed

0

Files Moved

0

Files Renamed

0

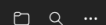
Folders Removed

1

Duplicates Found

Download Organized ZIP

Downloads



2cc6b081-752c-4e0e-a00f-dba0f0198f7d.zip

Open file

~~Screenshot_20-6-2026_161826_127.0.0.1.jpeg~~

Open file

~~Screenshot_20-6-2026_160010_127.0.0.1.jpeg~~

Open file

See more

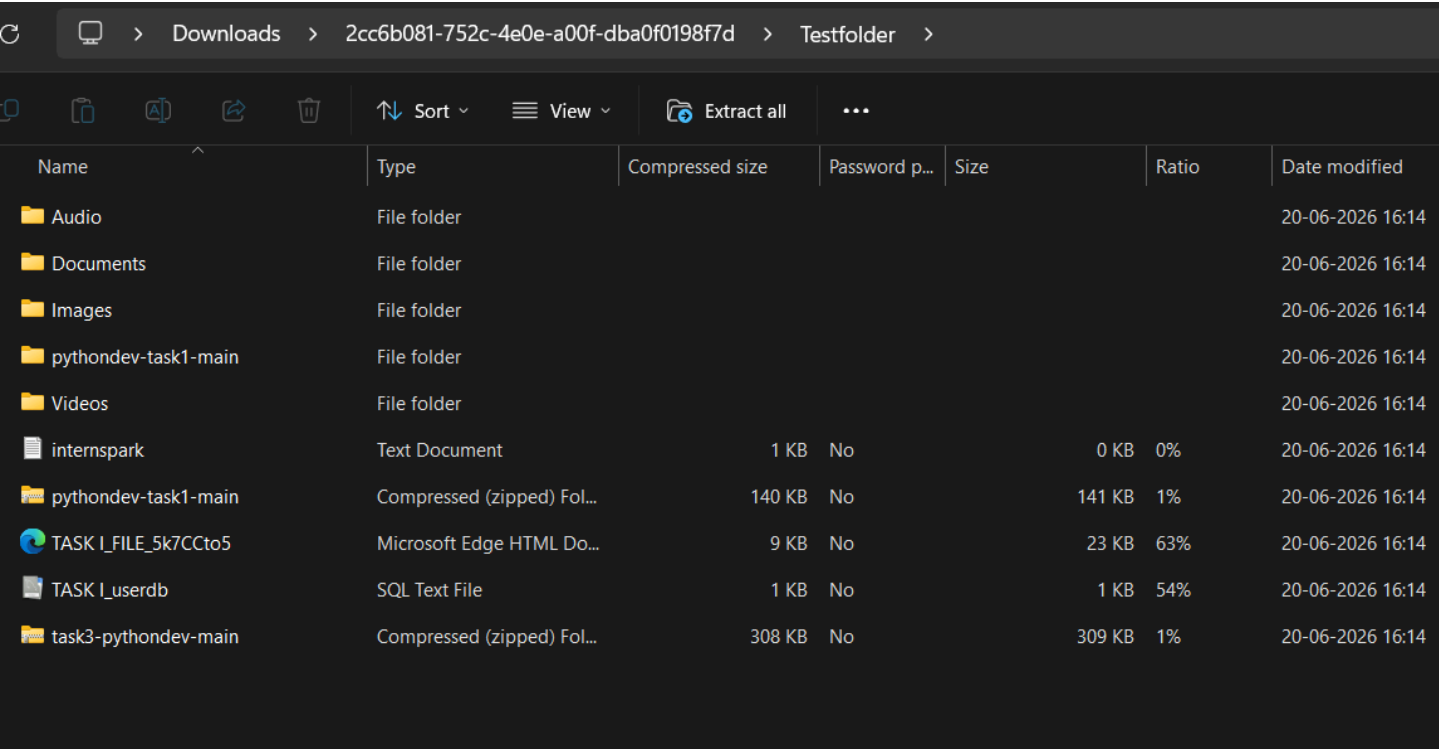
Smart File Organizer

A Flask automation tool that organizes uploaded ZIP files into structured folders.

⚡ Built by Madhiyarasu D D • 🚀 Flask Project • 📁 Internship Task

© 2026 Smart File Organizer

Organized Folder View



Submission Links

Item	Link
GitHub Repository	https://github.com/mathiarasan-art01/InternSpark-Internship
Hosted Application	https://smart-file-organizer-17sk.onrender.com

Notes

Run the application:

```
pip install -r requirements.txt
python app.py
```

Access the application:

http://127.0.0.1:5000

The system automatically initializes temporary directories for uploads and outputs during runtime. All files are processed securely in isolated environments to ensure cloud compatibility.

Learning Outcomes

- Flask-based file upload and automation system
 - Cloud-safe temporary file handling
 - ZIP extraction and compression techniques
 - File classification and organization algorithms
 - Duplicate detection using hashing
 - Deployment workflow using Render
 - Production-ready project structuring
-

Final Summary

The **Smart File Organizer (Cloud Version)** is a fully cloud-compatible automation system that eliminates local folder dependencies by using ZIP-based processing. It is deployable on Render and other cloud platforms while maintaining all automation features such as categorization, renaming, duplicate detection, and cleanup.

This project demonstrates practical cloud automation, secure file handling, and production-level Python development.